

1. INTRODUCTION & OVERVIEW

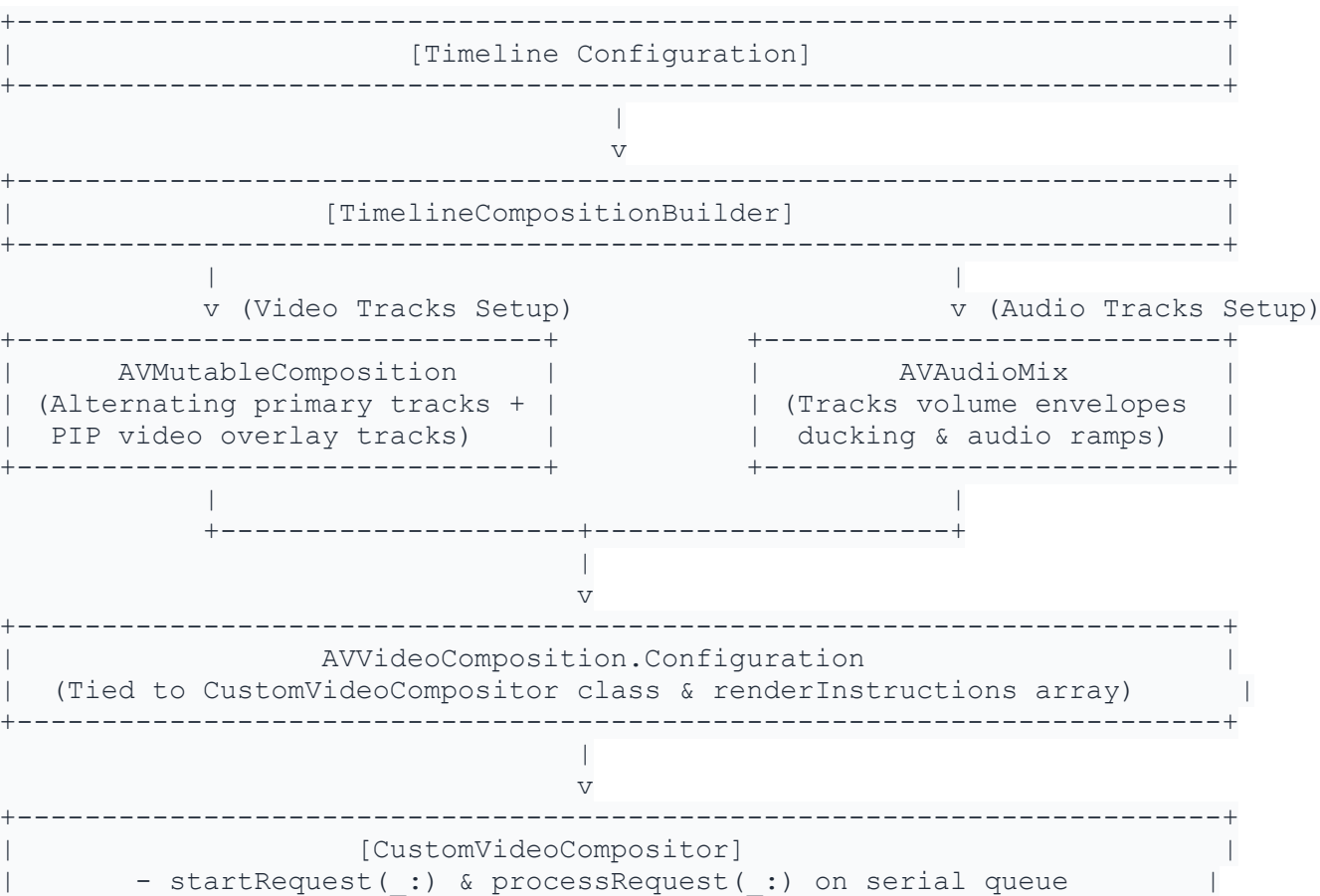
1.1. What the Video Editor Engine Is

The Video Editor Engine is a high-performance, frame-accurate rendering and composition pipeline designed for Apple platforms (iOS, macOS). Built on top of AVFoundation, CoreImage, and Metal, the engine allows developers to construct complex non-linear video editing sessions. It supports dynamic track layering, frame transitions, Picture-in-Picture (PIP) video overlays, vector sticker and text layouts, custom CoreImage color filter adjustments, and 3D Lookup Tables (LUTs).

1.2. High-Level Rendering Pipeline

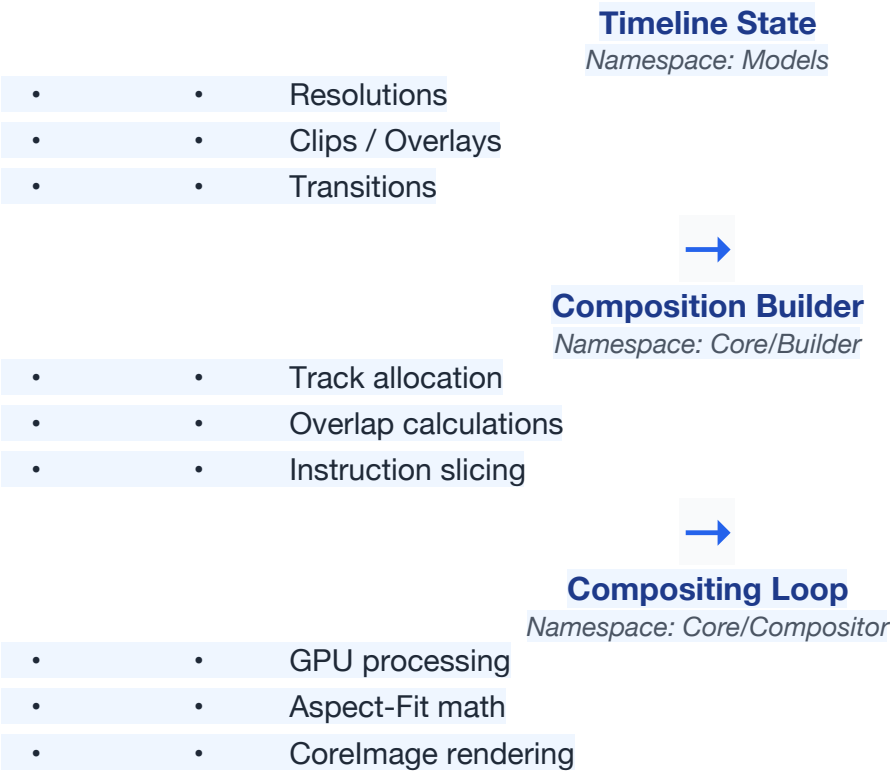
The engine operates on a pull-based asynchronous rendering loop. High-level project specifications are compiled into hardware-supported media tracks, sliced into logical instructions, and handed to a custom GPU-based compositor. The compositor intercepts decoded pixel buffers on-demand, passes them through vector orientation, scaling, and color pipelines, and writes the output back to AVFoundation's preview or encoding tracks.

1.3. Architecture Overview



and pass them inside composition instructions. Hardcoding IDs leads to track collision bugs.

2.2. Core Domain Data Flow



3. TIMELINE DATA MODELS

3.1. Class Specifications and Properties

+-----+ Timeline - canvasResolution: CanvasResolution - fps: Int - videoClips: [VideoClip] - audioClips: [AudioClip] - videoOverlays: [VideoOverlay] - stickerOverlays: [StickerOverlay] - textOverlays: [TextOverlay] - transitions: [Transition] +-----+	
+-----+ v +-----+	+-----+ v +-----+
+-----+ VideoClip - id: UUID - sourceURL: URL - timelineTimeRange: CMTimeRange - sourceTimeRange: CMTimeRange - transform: Transform +-----+	+-----+ VideoOverlay - id: UUID - sourceURL: URL - timelineTimeRange: CMTimeRange - sourceTimeRange: CMTimeRange - rect: CGRect (normalized) +-----+

- crop: CropArea		- scale: CGFloat	
- colorAdjustments: ColorAdjustments		- rotation: CGFloat	
- activeFilter: FilterType		- opacity: Float	
- activeLUT: LUTType		- blendMode: BlendMode	
+-----+		+-----+	

3.2. Properties Breakdown

- **Transform (Struct):** Defines translation offset (`translateX`, `translateY`), `scale` multiplier, `rotation` angle in radians, and layer `opacity`.
- **CropArea (Struct):** Declares cropped boundaries on the raw frame: `x`, `y`, `width`, and `height`, normalized as percentages (`0.0` to `1.0`).
- **ColorAdjustments (Struct):** Holds parameter modifiers: `brightness`, `contrast`, `saturation`, and `exposure`.

Why normalize coordinates?

Timeline items are designed to support changing resolution tiers dynamically (e.g. from a 720p preview to a 4K export). If positions were defined in absolute pixels, changing resolutions would displace overlays. Normalizing coordinates between `0.0` and `1.0` ensures overlays scale proportionally with any resolution tier.

4. COMPOSITION BUILDING

4.1. Track Allocation Flow

The builder compiles high-level models into structural tracks. Main clips are placed onto alternating tracks to support transition cross-decodes. Audio tracks are allocated dynamically only when audio components are detected, preventing export pipeline failures.

4.2. Alternating Tracks & Overlap Mathematics

When two clips overlap during a transition of duration D anchored at boundary time T , we adjust track insertion ranges:

- **Clip A (Track 1):** $Start_{tl}' = Start_{tl}, \quad Duration_{tl}' = Duration_{tl} + \frac{D}{2}$
 $Start_{src}' = Start_{src}, \quad Duration_{src}' = Duration_{src} + \frac{D}{2}$
- **Clip B (Track 2):** $Start_{tl}' = Start_{tl} - \frac{D}{2}, \quad Duration_{tl}' = Duration_{tl} + \frac{D}{2}$
 $Start_{src}' = Start_{src} - \frac{D}{2}, \quad Duration_{src}' = Duration_{src} + \frac{D}{2}$

4.3. Track Insertion Implementation

```
// Check transition at clip start
if let trans = transitions.first(where: { $0.atTime ==
clip.timelineTimeRange.start }) {
    let halfD = CMTimeMultiplyByFloat64(trans.duration, multiplier: 0.5)
    adjustedTimelineStart = clip.timelineTimeRange.start - halfD
    adjustedTimelineDuration = clip.timelineTimeRange.duration + halfD

    let newSrcStart = clip.sourceTimeRange.start - halfD
    if newSrcStart.seconds >= 0 {
        adjustedSourceStart = newSrcStart
        adjustedSourceDuration = clip.sourceTimeRange.duration + halfD
    }
}
```

Behind the Scenes: Decoders Overlapping

Hardware video decoders on mobile chips cannot decode two frames from the *same* track simultaneously. If we put overlapping clips on the same track, AVFoundation will drop frames during transitions. Alternating tracks forces the system to boot up two independent decoder streams, achieving smooth transitions.

5. TIMELINE SLICING ALGORITHM

5.1. Slice Slicing & Boundary Harvesting

To compile the rendering rules, we collect the start and end boundaries of all elements (clips, transitions, overlays, stickers, texts) into a chronological, sorted array of distinct timestamps. This divides the composition timeline into a series of contiguous, non-overlapping time slices:

```
Timeline:
|----- Slice 1 -----|---- Slice 2 ----|----- Slice 3 -----|
T=0                      T=2.0          T=4.25          T=5.75
(Clip 1 Active)          (Clip 1 + Text)  (Clip 1 + Clip 2 + Text)
```

5.2. Midpoint Evaluation Pattern

To identify which elements are active in a slice $[Start, End]$, we evaluate the status at the slice midpoint: $MidTime = \frac{Start + End}{2}$. Checking active states at $MidTime$ avoids timing precision collisions exactly at start or end bounds.

5.3. Slicing Algorithm Code

```
let sortedTimes = boundaryTimes.filter { $0 <= maxTime }.sorted()
for i in 0.. $(sortedTimes.count - 1)$  {
    let startSec = sortedTimes[i]
    let endSec = sortedTimes[i+1]
    if endSec - startSec < 0.01 { continue } // Filter sub-frame noise
```

```

let sliceRange = CMTimeRange(
    start: CMTime(seconds: startSec, preferredTimescale: 600),
    end: CMTime(seconds: endSec, preferredTimescale: 600)
)
let midTime = CMTime(seconds: (startSec + endSec) / 2.0, preferredTimescale:
600)

// Evaluate active clips, overlays, stickers, and texts at midTime
// Instantiate CustomVideoCompositionInstruction
}

```

6. RENDERING PIPELINE

6.1. Frame-by-Frame Execution Loop

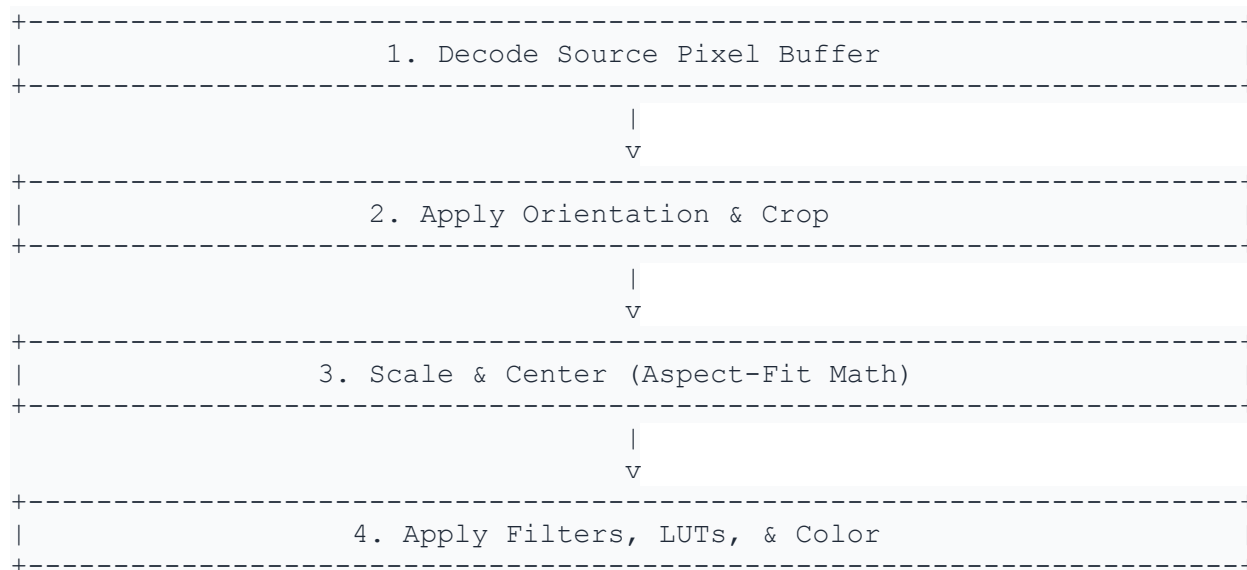
For each frame, the compositor receives an `AVAsynchronousVideoCompositionRequest` on a serial priority queue. It allocates an output pixel buffer, gathers the decoded source track buffers, and processes the graphics pipeline via CoreImage.

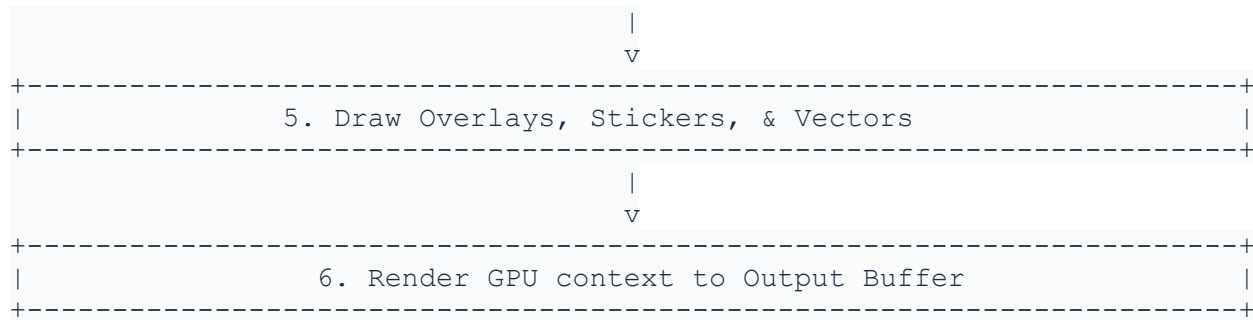
6.2. Aspect-Fit Math

To fit clip buffers of size (W_s, H_s) onto the canvas size (W_c, H_c) without distortion:

- Compute scales: $Scale_w = W_c / W_s$, $Scale_h = H_c / H_s$
- Uniform aspect-fit scale: $Scale_{fit} = \min(Scale_w, Scale_h)$
- Centering translations: $T_x = \frac{W_c - (W_s \times Scale_{fit})}{2}$,
 $T_y = \frac{H_c - (H_s \times Scale_{fit})}{2}$

6.3. Blending and Graphic Rendering





Why use Metal-backed CIContext?

Instantiating a new `CIContext` for every frame leaks memory and causes rendering times to exceed the 16ms frame window (for 60 FPS). We cache a single Metal-backed `CIContext`, enabling zero-copy memory transfers directly inside GPU buffers.

7. AUDIO PIPELINE

7.1. Audio Track Allocation & Mixing

The audio pipeline processes multiple tracks in parallel to the video rendering pipeline. While the GPU renders video, audio buffers are read from composition tracks, mixed based on volume envelopes, and compressed to AAC files during export.

7.2. AVAudioMix Volume Envelopes

Volume ramping (ducking background music during overlays or fading sound in/out) is compiled using `AVAudioMixInputParameters`:

```

let audioParams = AVMutableAudioMixInputParameters(track: compAudioTrack)
// Constant volume
audioParams.setVolume(1.0, at: .zero)
// Ducking volume ramp (fade down background music over timeline range)
audioParams.setVolumeRamp(fromStartVolume: 1.0, toEndVolume: 0.15, timeRange:
duckRange)

```

7.3. Audio Pipeline Specifications

Pipeline Stage

Operation & Mechanics

Target Values / Precision

Audio Decoding

Reads AAC/MP3 files and decodes to raw uncompressed sample blocks.

Linear PCM (32-bit Float format)

Sample Rate Conversion

Resamples differing asset streams to align with timeline properties.

Typical Target: **48,000 Hz** (Pro Video)

Audio Mixing

Applies volume ramp gains and mixes channels.

Stereo output downmix

Audio Encoding

Compresses PCM mix back to audio track package.

AAC codec at 320 kbps bitrate

8. EXPORT PIPELINE

8.1. AVAssetExportSession Configuration

For high-quality exports, we configure `AVAssetExportSession` with a highest quality preset and optimize the file layout for fast start streaming:

```
let exportSession = AVAssetExportSession(
    asset: composition,
    presetName: AVAssetExportPresetHighestQuality
)
exportSession.outputURL = outputURL
exportSession.outputFileType = .mp4
exportSession.shouldOptimizeForNetworkUse = true
await exportSession.export()
```

8.2. Network Optimization (Fast Start)

Setting `shouldOptimizeForNetworkUse = true` rearranges the structure of the MP4 file by moving the metadata index atom (`moov`) to the beginning of the file, preceding the frame data atom (`mdat`):

Non-Optimized Layout (moov at end):

+-----+-----+-----+-----+			
ftyp header	mdat atom	moov atom	
+-----+-----+-----+-----+			
	(Raw Frames)	(Metadata)	

Optimized Layout (moov at start):

+-----+-----+-----+-----+			
ftyp header	moov atom	mdat atom	
+-----+-----+-----+-----+			
(Metadata)	(Raw Frames)		

Behind the Scenes: Moov Relocation

When relocating the `moov` atom, the absolute byte positions of the media chunks inside `mdat` are shifted forward. The exporter automatically updates the index table tables (`stco` / `co64` atoms) by adding the size of the shifted `moov` atom to all offset indices. Without this shift, the video data offsets would be invalid, resulting in a corrupted video file.

9. COREMEDIA TIME SYSTEM

9.1. Rational Time Math Specification

CoreMedia represents time as a rational number struct (`CMTime`) instead of a floating-point number, preventing rounding errors from accumulating over editing timelines:

$$\text{Time (Seconds)} = \frac{\text{value}}{\text{timescale}}$$

9.2. Preferred Timescale Denominator

When instantiating a `CMTime`, we specify a `preferredTimescale`. Choosing the correct denominator is critical for editing stability:

Timescale

Application

Rationale

600

Timeline and Video Clips

The common denominator for standard video framerates. Divisible by 24, 25, 30, 50, and 60 FPS, ensuring frame markers sit on integer boundaries.

Example: 1 frame at 24 FPS → $\frac{25}{600}$ seconds.

44100 / 48000

Audio Tracks

Matches physical audio sampling rates, preventing sub-sample drift and audio phase clicks.

1000000000

System Clock Times

Nanosecond precision for system clock sync.

Why does floating-point math fail in video editing?

At 60 FPS, each frame lasts $0.0166666\dots$ seconds. Summing floating-point representations of this interval over a 10-minute video introduces significant rounding errors, shifting frame boundaries by up to half a second, resulting in dropped frames or audio-video desync. Using a rational timescale of 600 ensures that every frame sits on an exact integer tick (e.g. 10 ticks per frame).

10. COORDINATE SYSTEMS & TRANSFORMATIONS

10.1. Mismatched Layout Spaces

Video rendering involves mapping three mismatched coordinate systems:

- **Raw Source Space:** Pixel coordinates (W_s, H_s) with origin at the top-left, rotated based on device camera metadata.
- **UIKit UI Space:** Normalized coordinates (0.0 to 1.0) with origin at the top-left, where y increases downwards.

- **CoreImage Cartesian Space:** Pixel coordinates with origin at the bottom-left, where y increases upwards.

10.2. Transformation Graph

1. UIKit Canvas Space

```
(0,0) (Top-Left)
+-----> X (0.0 to 1.0)
| Subtitle (y=0.82)
v Y (0.0 to 1.0)
```



Transformation Phase

Transforms vertical origin

$$\text{ciY} = 1.0 - y_{\text{ui}} - h_{\text{ui}}$$


2. CoreImage Space

```
^ Y (0.0 to Height_c)
| Text center (ciY)
+-----> X (0.0 to Width_c)
(0,0) (Bottom-Left)
```

10.3. Coordinate Conversion Code

```
// Translate top-left normalized UIKit space to bottom-left CoreImage space
let ciY = 1.0 - overlay.rect.origin.y - overlay.rect.height
let overlayCenter = CGPoint(
    x: canvasSize.width * overlay.rect.origin.x + (canvasSize.width *
overlay.rect.width) / 2.0,
    y: canvasSize.height * ciY + (canvasSize.height * overlay.rect.height) / 2.0
)
```

11. CORE IMAGE FILTERS REFERENCE

11.1. Parameter Ranges and Recommendations

Filter Name & Key

Constant Key Identifier

Valid Value Range

Default Value

Significance & Visual Impact

Brightness

CIColorControls

kCIInputBrightnessKey

(inputBrightness)

-1.0 to 1.0

0.0

Adjusts the overall luminance offset. Increasing shifts all pixel values up, whitening shadows, while decreasing darkens the scene.

Contrast

```
CIColorControls  
kCIInputContrastKey  
(inputContrast)  
0.0 to 4.0  
1.0
```

Alters the luminance slope. At 0.0, the output is a flat grey frame. Values above 1.0 make highlights brighter and shadows darker, increasing color separation.

Saturation

```
CIColorControls  
kCIInputSaturationKey  
(inputSaturation)  
0.0 to 2.0  
1.0
```

Controls the color intensity. Setting to 0.0 produces a monochrome (black & white) image. Setting to 2.0 creates vivid, highly saturated tones.

Exposure

```
CIColorAdjust  
kCIInputEVKey  
(inputEV)  
-10.0 to 10.0  
0.0
```

Adjusts the brightness using a logarithmic exposure value (stops). Each unit increases or decreases the exposure by a factor of 2. Useful for highlights correction.

Opacity

```
CIColorMatrix  
inputAVector  
(Vector matrix multiplier)  
0.0 to 1.0  
1.0
```

Sets frame transparency. Evaluated using a vector translation of the alpha channel: `CIVector(x:0, y:0, z:0, w:opacity)`. Value 0.0 makes the clip invisible, revealing tracks underneath.

Sepia Intensity

```
CISepiaTone  
kCIInputIntensityKey  
(inputIntensity)  
0.0 to 1.0  
1.0
```

Determines the strength of the warm sepia tone filter. A value of 0.8 is applied in our engine for a classic warm style.

Blur Radius

```
CIGaussianBlur  
kCIInputRadiusKey  
(inputRadius)  
0.0 to 100.0  
10.0
```

Specifies the standard deviation of the Gaussian distribution in pixels. Increasing it results in a heavier blur. Our engine applies 12.0 for cinematic background separation.

Vignette Intensity

```
CIVignette  
kCIInputIntensityKey
```

```
(inputIntensity)
-1.0 to 1.0
0.0
```

Sets the dark/light falloff intensity at the borders. Positive values darken the screen edges, creating focal focus. Our engine applies `1.0`.

Vignette Radius

```
CVignette
kCIInputRadiusKey
(inputRadius)
0.0 to 30.0
1.0
```

Adjusts the vignette radius fall-off range from the center. Our engine sets this to `15.0` to smooth vignette borders over high-resolution canvases.

12. SYSTEM ARCHITECTURE & CLASS RESPONSIBILITIES

12.1. Class Responsibility Specifications

- **TimelineCompositionBuilder**: Compile non-linear projects into track-level structures. Schedules tracks, sets transition times, and generates instruction lists.
- **CustomVideoCompositor**: Conforms to `AVVideoCompositing`. Manages the execution serial queue and retrieves pixel buffers from requested source tracks.
- **ClipEffectsProcessor**: Normalizes rotation matrices, applies aspect fit conversions, processes color control filters, Vignette/Sepia styles, and 3D LUT look-up tables.
- **VideoOverlayProcessor**: Position overlay frames onto base clips and blends layers using blending filters.
- **TextOverlayProcessor**: Compiles string layers to graphic textures and positions titles dynamically relative to base resolution sizes.
- **StickerOverlayProcessor**: Renders system vector symbol layouts (SF Symbols).

12.2. Frame Rendering Walkthrough Timeline

T = 0ms	T = 4ms	T = 12ms
T = 16ms		
[startRequest(_)]	-----> [Fetch Source Buffers]	----> [CoreImage Processing]
	----> [Write & Finish]	
- Receives compositing request	- Fetches fromTrackBuffer	- PreferredTransform check
- Renders final CIImage to		
- Spawns background thread	- Fetches toTrackBuffer	- Aspect-Fit layout scaling
	destination buffer	

```
- Reads active instructions      (If transition is active)      - Blends overlays &
texts      - request.finish() called
```

13. INTERVIEW NOTES & REFERENCE GUIDE

13.1. Technical Interview Questions & Answers

Tricky Edge Case

Q: Why does the AVAssetExportSession crash with "Operation Stopped" when exporting compositions containing empty tracks?

Answer: If you add tracks (audio or video) to a composition but do not write any segments to them, the track timing tables are empty. When the exporter parses the timing boundaries, it fails to resolve offsets and throws a generic "Operation Stopped" error. The solution is to compile tracks dynamically (on-demand) only when source assets contain valid media channels.

Tricky Edge Case

Q: Why do stickers and text layers appear upside down when drawn inside custom compositors?

Answer: This is caused by coordinate space orientation mismatches. UIKit/AppKit has its origin (0,0) at the top-left (y-axis increases downwards), whereas CoreImage/GPU buffer systems place the origin at the bottom-left (y-axis increases upwards). To resolve this, you must transform UIKit coordinates to CoreImage coordinates using: $y_{ci} = 1.0 - y_{uikit} - height_{uikit}$, and translate the graphics context to the target center location before drawing.

Tricky Edge Case

Q: Why does a hardcoded title font size (e.g., 24pt) look correct on 720p simulator previews but appear microscopic on a 4K exported video?

Answer: Hardcoded point sizes represent absolute pixel counts. A height of 24px is 3.3% of a 720p frame, but only 1.1% of a 4K frame. When the 4K output is scaled down to fit simulator preview widgets, the text shrinks to less than 3 screen pixels. To resolve this, scale text font sizes dynamically relative to the target height of the bounding box: $FontSize = BoundingBox_{height} \times 0.7$.

14. APPENDICES

14.1. Core Mathematical Formula Reference

- **Transition Overlap (Clip Start - Clip B):** $Start'_{tl} = Start_{tl} - \frac{D}{2}, \quad Duration'_{tl} = Duration_{tl} + \frac{D}{2}$
- **Transition Overlap (Clip End - Clip A):** $Start'_{tl} = Start_{tl}, \quad Duration'_{tl} = Duration_{tl} + \frac{D}{2}$
- **Aspect-Fit Center Translation:** $T_x = \frac{W_c - (W_s \times Scale_{fit})}{2}, \quad T_y = \frac{H_c - (H_s \times Scale_{fit})}{2}$
- **UIKit to CoreImage Y-Coordinate Flip:** $y_{ci} = 1.0 - y_{uikit} - height_{uikit}$

14.2. CoreMedia Time Rational Representation

$\text{Time (Seconds)} = \frac{\text{value (Numerator: Ticks)}}{\text{timescale (Denominator: Ticks per Second)}}$

Using a preferred timescale of **600** ensures that fractional frame positions map to clean integers without float rounding drift.